Model Manual

Stationary Schrodinger Equation

Elmer FEM 9.0 — SchrodingerSolve

Module: `SchrodingerSolve` · **Subroutine:** `SchrodingerSolver`

Module author: UFPA project (see *How to cite*) · **Document author:** UFPA project

Elmer version: 9.0

Contents

1. Introduction
2. Theory
3. Boundary conditions
4. The potential V(r)
5. Keywords
6. Units and conventions
7. Analytical reference solutions
8. Limitations
9. How to cite

---

1. Introduction

This module solves the **stationary (time-independent) Schrödinger equation**, the fundamental eigenvalue problem of quantum mechanics:

$$-\frac{\hbar^2}{2m}\nabla^2\psi + V(\mathbf{r})\,\psi = E\,\psi$$

The module computes the `N` lowest eigenvalues `E` (the energies) and the associated eigenfunctions `psi` (the wave functions). It is intended for:

- potential wells (particle in a box, finite well)
- harmonic oscillators
- molecular potentials (Morse)
- double wells (tunnelling)
- periodic potentials (band structure)

The module works in **1D, 2D and 3D** with a single implementation; the dimension is taken from the `Coordinate System` declared in the `Simulation` section.

Note: *for the* time-dependent *problem (nonlinear Schrödinger equation, wave-packet propagation) this module does not apply — it solves stationary states only.*

---

2. Theory

2.1 Weak form

The equation is discretised by the **Galerkin finite element method**.
Expanding the solution in the basis of shape functions `Nᵢ`,

$$\psi(\mathbf{r}) = \sum_i N_i(\mathbf{r})\,\psi_i$$

and testing with `Nⱼ` (integrating the Laplacian term by parts) gives

$$\underbrace{\int_\Omega \nabla N_j \cdot \nabla N_i \, d\Omega}_{K_{ji}}\,\psi_i + \underbrace{\int_\Omega N_j\, V\, N_i \, d\Omega}_{M^a_{ji}}\,\psi_i = E\,\underbrace{\int_\Omega N_j N_i \, d\Omega}_{M_{ji}}\,\psi_i$$

that is, the **generalised eigenvalue problem**:

$$A\,\psi = E\,M\,\psi, \qquad A = K + M^a$$

where

Matrix	Name	Expression	Meaning
`K`	stiffness	`∫ ∇Nᵢ·∇Nⱼ dΩ`	kinetic energy
`M`	mass	`∫ NᵢNⱼ dΩ`	inner product (normalisation)
`Mᵃ`	weighted mass	`∫ Nᵢ V Nⱼ dΩ`	potential energy

2.2 Solution of the eigenproblem

The generalised eigenproblem is solved by the **LAPACK `DSYGV`** routine, which is appropriate for symmetric matrices with positive-definite `M`. The `N` lowest eigenvalues are sorted and reported.
Dirichlet boundary conditions are imposed by **eliminating the fixed degrees of freedom**, rather than by the common trick of setting the diagonal to 1 (which introduces spurious eigenvalues).

2.3 Quadrature

The integrals use **Gauss quadrature** at the element quadrature points, with `N` points per direction. This means the potential `V` is evaluated at the Gauss points and integrated exactly, rather than interpolated approximately from the nodal values — which matters for rapidly varying potentials.

---

3. Boundary conditions

3.1 Homogeneous Dirichlet (default)

```
Apply Dirichlet = Logical True
```

Nodes belonging to the geometric boundary of the mesh are detected **automatically** and assigned $\psi = 0$. This corresponds to an infinite wall (infinite well) and is the appropriate condition for most bound-state cases. It is **not necessary** to declare a `Boundary Condition` section in the SIF.

3.2 Automatic detection

The solver identifies boundary nodes from the mesh geometry (coordinates at the domain extremes in each direction), not from the list of boundary elements. This makes the implementation robust with respect to externally generated meshes.

3.3 Normal direction

The flux $\nabla\psi\cdot\hat{n}$ vanishes naturally wherever no Dirichlet condition is imposed (homogeneous Neumann condition), which corresponds to a reflecting wall.

> ***Important limitation:*** *this module implements* ***no radiation condition*** *(*Sommerfeld*) and no* complex scaling*. It therefore* ***cannot*** *be used to compute transmission coefficients through barriers or resonant states. That would require an infinite-domain treatment (see `00_planejamento/referencias/` — the `qsolver` program, CPC 329 (2026) 110393).*

---

4. The potential V(r)

The potential is read as a **nodal field** and accepts three forms, in order of precedence.

4.1 `Potential` in `Body Force` (recommended)

1D — a single variable; `tx` holds the coordinate value:

```
Body Force 1
  Potential = Variable Coordinate 1
    Real MATC "1.0*tx*tx"
End
```

2D/3D — multiple variables separated by a **comma**; `tx` is the first and `tx(1)`, `tx(2)` are the following ones:

```
Body Force 1
  Potential = Variable Coordinate 1, Coordinate 2, Coordinate 3
    Real MATC "0.25*(tx*tx + tx(1)*tx(1) + tx(2)*tx(2))"
End
```

> ***The separator is a COMMA, not a space.*** *`Variable Coordinate 1 Coordinate 2` is interpreted as a* ***single*** *non-existent variable name. The reason is in the Elmer parser (`Lists.F90, ListParseStrToVars): l1 = INDEX( str(l0:slen), ',' )`. See `01_simulacoes/schrodinger_P1_2/docs/SINTAXE_MATC_POTENCIAL.md`.*

4.2 `Potential X` / `Potential Y` / `Potential Z`

Component-wise construction, combined internally by the solver (`V = V_x + V_y + V_z`). Each term is a 1D expression:

```
Body Force 1
  Potential X = Variable Coordinate 1
    Real MATC "0.25*tx*tx"
  Potential Y = Variable Coordinate 2
    Real MATC "0.25*tx*tx"
  Potential Z = Variable Coordinate 3
    Real MATC "0.25*tx*tx"
End
```

Useful when the terms come from different sources, or to avoid the multidimensional MATC syntax.

4.3 Absent

If none of the forms above is present, `V(r) = 0` (infinite well).
The solver **reports in the log** which path was taken, so that `V = 0` is never assumed silently:

```
SchrodingerSolver: V(x) read as nodal field from Body Force.
SchrodingerSolver:   V_min =   0.000000E+00   V_max =   5.000000E+01
```

*The `V_min/V_max` log line is the quickest check that the potential was read correctly. When the `Potential X/Y/Z` form is used it is reported **per component**, and summed internally.*

---

5. Keywords

5.1 `Simulation` section

Keyword	Type	Description
`Coordinate System`	String	`Cartesian 1D`, `Cartesian 2D` or `Cartesian 3D`. **Must match**
`Simulation Type`	String	`Steady State`
`Steady State Max Iterations`	Integer	`1`
`Max Output Level`	Integer	Use **≥ 4** to see the solver messages (it reports at `Level=`

5.2 `Body Force` section

Keyword	Type	Description
`Potential`	Real (field)	Potential `V(r)`. Nodal field or MATC expres
`Potential X`, `Potential Y`, `Potential Z`	Real (field)	Potential components. See §4.2.

5.3 `Solver` section

Keyword	Type	Default	Description
`Equation`	String	—	Equation name (e.g. `"Schrodinger"`)
`Procedure`	String	—	`"SchrodingerSolver" "SchrodingerSolver"`
`Variable`	String	`psi`	Name of the output field (the wave function)
`Variable DOFs`	Integer	`1`	Degrees of freedom per node. Keep at 1.
`Eigen System Values`	Integer	`5`	**Number of eigenvalues to compute** (the `N`
`Apply Dirichlet`	Logical	`True`	Impose `ψ = 0` on boundary nodes (§3)
`Linear System Solver`	String	`Direct`	Keep `Direct`
`Linear System Direct Method`	String	—	`"umfpack"` or `"mumps"` (if the build has MUMPS
`Exportar Matrizes`	Logical	`False`	Write `A` and `M` as ASCII triplets (§5.4)
`Exportar Prefixo`	String	`schrod`	Prefix of the exported files

5.4 Matrix export (`Exportar Matrizes`)

When `Exportar Matrizes = Logical True`, the solver writes:

File	Content	Format
`<prefix>_A.dat`	operator $A = K + M^a$	`n n nnz` then `i j value` (**0-based**)
`<prefix>_M.dat`	mass matrix `M`	idem
`<prefix>_fixed.dat`	Dirichlet DOFs	count, then the indices (**0-based**)

Purpose: it allows the eigenproblem to be solved **outside Elmer**, with a sparse solver (`scipy.sparse.linalg.eigsh`, ARPACK shift-invert) or a parallel one (PETSc/SLEPc). This is necessary because `DSYGV` is **dense and serial** (memory $O(n^2)$, cost $O(n^3)$), and because Elmer 9.0 **cannot partition a mesh** on this platform.

Solving the exported system:

```
python3 01_simulacoes/schrodinger_P1_2/scripts/resolve_exportado.py schrod 5
```

See `01_simulacoes/schrodinger_P1_2/docs/PARALELISMO.md` for the full analysis.

5.5 VTU output

To generate the `.vtu` file (ParaView), add a second solver:

```
Solver 2
  Exec Solver = "After Simulation"
  Equation = "ResultOutput"
  Procedure = "ResultOutputSolve" "ResultOutputSolver"
  Output File Name = File "run"
  Output Format = String "vtu"
End
```

and activate both: `Active Solvers(2) = 1 2`.

The file is written to **`malha/run_t0001.vtu`** (Elmer writes into the mesh directory) and contains the mesh plus the **`psi`** field (the **mode 1** eigenfunction).

---

6. Units and conventions

The solver works with

$$\hbar = 1, \qquad m = 1$$

so the implemented equation is

$$-\nabla^2\psi + V(\mathbf{r})\,\psi = E\,\psi$$

⚠ Relation to the usual form

The form found in many texts (and in the Elmer manual for `HelmholtzSolver`) is

$$-\tfrac{1}{2}\nabla^2\psi + V\psi = E\psi$$

This is different. The conversion is:

$$E_{\text{usual}} = \tfrac{1}{2}\,E_{\text{here}} \qquad \Longleftrightarrow \qquad E_{\text{here}} = 2\,E_{\text{usual}}$$

In atomic units the solver effectively adopts `m* = 1/2`, i.e. **twice** the electron mass. To convert to a concrete physical system, use the full expression

$$E = \frac{\hbar^2}{2m} \times E_{\text{dimensionless}}$$

with the `E_dimensionless` reported by the solver.

> ***Why this matters:*** *the initial validation of the project used the infinite well with `L = 40`, where **the two conventions coincide by numerical accident** — the test could not distinguish them. The convention was only settled with tests that **do** distinguish the forms: infinite well with `L = 10` and the harmonic oscillator. See `01_simulacoes/schrodinger_P1_2/docs/VALIDACAO_POTENCIAL.md`.*

---

7. Analytical reference solutions

Formulas **in this solver's convention** (`−∇²ψ + Vψ = Eψ`). All verified numerically (see `01_simulacoes/RESULTADOS_TODOS_CASOS.md`).

Potential	Eigenvalues	Note
Infinite well 1D, `[0,L]`	`E_n = (nπ/L)², n = 1,2,…`	
Infinite well, `d`-dimensional, side `L`	`E = (π²/L²)Σᵢ nᵢ²`	degeneracy by perm
Isotropic harmonic oscillator `V = α r²`	`E_n = 2√α (n + d/2)`	degeneracy `(n+d−1)!`
Morse potential	well minimum `= −Dₑ` (check on `V`)	zero-point energy ≈
Double well `V = a(x²−b)²`	splitting `E₁−E₀ > 0`	measures tunnelling
Periodic potential `V = A cos(kx)`	bands	see §8

---

8. Limitations

Stated explicitly to prevent misuse.

Limitation	Detail
Finite domain	The domain must be truncated with `ψ = 0`. Excited states of non-bi
No radiation condition	Cannot compute barrier transmission/reflection or resonant states
Dense and serial	`DSYGV` is `O(n²)` in memory and `O(n³)` in cost. In practice ~8000 DOF
No MPI	The solver is **not MPI-aware**: it assembles **dense local** matrices.
Mode 1 only in the VTU	Attempts to publish all modes via `VariableAdd` caused heap corrupt
`m = 1` convention	See §6.

---

9. How to cite

```
SchrodingerSolve — a finite element module for the stationary Schrödinger
equation, Elmer FEM 9.0. NLSE/Schrodinger project, UFPA.
```

Related documents:

- `Manual_Solver_Schrodinger.md` — implementation, compilation and usage
- `01_simulacoes/schrodinger_P1_2/docs/` — validation and syntax
- `01_simulacoes/schrodinger_P1_2/casos/<case>/TUTORIAL.md` — tutorials

Last updated: 2026-10-08